



INVENTIX · TEAM 4

Database build report

What was built, what was added beyond the specification, what was run, and what the verification actually proved.

Source docs/dbreport.md

Companion 09-database-build.md

Generated 3 September 2026

Contents

1. In one paragraph

2. What now exists

3. The new table

0016_supplier_ranking.sql — not in specification §5

4. Three columns beyond §5.5

5. Everything else added that §5 does not mention

A trigger on every table

Constraints that carry a rule

A role guard a foreign key cannot express

50 indexes

A sequence

6. A contradiction in the specification, resolved

7. What was run

About the shim

8. What the verification proved

Every file applies, in order

Row level security actually works

The functions do what they claim

The demo data loads and behaves

The added constraints hold

One table is deny-all, on purpose

9. What was not done

10. How to apply it to Supabase

11. Documents updated

12. Where this leaves the project

What was built, what was added beyond the specification, what was run, and what the verification actually proved. Everything here is a record of work already done, not a plan.

1. In one paragraph

Every stub in `database/` is now real SQL: 31 files, 1,373 lines. That covers all 15 tables from specification \$5, one table the specification does not have, two SQL functions, nine row level security policy files and four seed files. The whole schema was then executed against a real PostgreSQL 16 server, and the six-step security procedure from `docs/09-database-build.md` was run rather than assumed. Every check passed. Nothing has been applied to Supabase yet — that is your step, and section 8 says how.

2. What now exists

Folder	Files	Lines	Contents
<code>migrations/</code>	16	672	Every table, its constraints, its indexes and its trigger
<code>functions/</code>	2	152	<code>apply_stock_adjustment</code> , <code>recompute_supplier_ranking</code>
<code>policies/</code>	9	274	Row level security — the part that actually protects the data
<code>seeds/</code>	4	275	Config, 36 catalog products, 5 festivals, demo data

Measured on the live server after applying all of it:

Object	Count
Tables	16
Indexes	50
Functions	7
RLS policies	24
Triggers	16
Catalog products seeded	36
Config keys seeded	10
Seasonal events seeded	5

Seven functions rather than two: the two in `functions/` , plus `set_updated_at()` , `auth_role()` , `assert_supplier()` , `supplier_listings_role_guard()` and `app_config_touch()` , all of which are defined inside the migration or policy file that needs them.

3. The new table

This is the part you asked to be told about explicitly.

0016_supplier_ranking.sql — not in specification §5

One row per supplier, rewritten daily by `recompute_supplier_ranking()`.

Column	Holds
<code>supplier_id</code>	Primary key, references <code>profiles</code>
<code>score</code>	0–100. Null until the job has run
<code>average_rating</code> , <code>rating_count</code>	The quality component and its confidence
<code>measured_delivery_days</code> , <code>completed_orders</code>	The speed component and its confidence
<code>is_new_supplier</code>	Below the configured minimum of completed orders
<code>computed_at</code>	When the job last touched this row

Why it had to be added. §12 defines a weighted ranking and §14 requires a daily job that "recomputes supplier ranking scores so search results do not have to compute them live". A score therefore has to be stored somewhere, and no table in §5 holds one. Without this table, every supplier search would have to aggregate every order and rating in the system on the spot — which is precisely what §14 rules out.

What was considered instead. A database *view* was the obvious alternative and was rejected: a view re-runs its query on every read, so it is the live computation under a different name. Computing in Python on each search was rejected for the same reason, plus it would pull every order and rating into the application just to average them.

What it deliberately does not hold. Only two of the four §12.1 components are in the score — quality rating (40%) and measured delivery speed (30%), renormalised to 100. Availability (20%) and price (10%) depend on *what the customer is currently ordering*, so they cannot be precomputed; `backend/app/domain/ranking.py` applies them per query. This is why a supplier can rank second for rice and fifth for sugar while having one stored score.

Recorded as D-006 in `docs/04-decision-log.md`.

4. Three columns beyond §5.5

All three on `orders`. Recorded as D-007.

Column	Why
<code>reference</code>	The <code>DEL-0001</code> your order cards already display. <code>OrderSummary.reference</code> existed in the frontend with nothing behind it. Defaulted from a sequence so an existing order's reference never changes when another row is deleted.
<code>idempotency_key</code>	§15.4 requires sending to be idempotent. Nullable and unique, so a replayed request fails on the constraint and the backend returns the original order instead of creating a second one.
<code>notes</code>	The restock popup collects notes per §6.5 and had nowhere to store them.

`idempotency_key` deserves a note on what was avoided. The obvious design is a separate `idempotency_keys` table. A nullable unique column enforces exactly the same guarantee with no extra table, no extra join, and no second thing to clean up — and it puts the constraint on the row it protects.

5. Everything else added that \$5 does not mention

A trigger on every table

`updated_at timestamptz default now()` fires **only on insert**. SQL has no "on update" for a default, so without a trigger every `updated_at` column in the schema would have been permanently wrong — a stock item edited fifty times would still claim it was last touched the day it was created. No error, no warning, just a column that quietly lies.

`set_updated_at()` is defined at the top of `0001_profiles.sql`, because that is the first file to run and every table below it needs the function to exist. Fifteen tables attach it; `stock_adjustments` deliberately does not, because an audit row is never edited.

Constraints that carry a rule

Written as constraints rather than left to service code, because a rule in Python runs only when Python runs, while a constraint runs always — including when someone types SQL into the Supabase editor by hand.

Constraint	Stops
<code>check (role in ('customer','supplier'))</code>	An unknown role, which would break every policy downstream
<code>check (role = 'supplier' or delivery_areas is null)</code>	A customer holding supplier-only data
<code>check (status in (...eight values...))</code>	An order in a state the state machine has never heard of
<code>check (status <> 'rejected' or rejection_reason is not null)</code>	\$10.2's required reason being skipped
<code>unique (customer_id, file_hash)</code>	The same sales report being applied twice (§15.4)
<code>unique (owner_id, catalog_product_id)</code>	Two stock rows for one product in one shop
<code>unique (supplier_id, catalog_product_id)</code>	Two listings for one product from one supplier
<code>unique (order_id) on ratings</code>	More than one rating per order
<code>check (quality_score between 1 and 5)</code>	A six-star rating quietly inflating the ranking
<code>check (customer_id <> supplier_id)</code>	An order to yourself
<code>check (quantity_on_hand >= 0)</code>	Negative stock

A role guard a foreign key cannot express

`supplier_listings.supplier_id` references `profiles`, which proves the profile exists but **cannot prove it is a supplier** — a foreign key has no opinion about the contents of the row it points at. A trigger calling `assert_supplier()` closes that gap. The same reasoning would apply to `stock_items.owner_id`; it is not guarded yet, and section 9 lists it.

50 indexes

PostgreSQL indexes primary keys and unique constraints automatically. It does **not** index foreign keys, and nearly every query the backend runs filters on one. Two are worth calling out:

- `supplier_listings (catalog_product_id) where is_active` — a **partial** index. Product search only looks at active listings, so indexing the inactive ones would waste space and slow writes.
- `orders (customer_id, status)` and `orders (supplier_id, status)` — separate, because the two feeds read the same table from opposite ends.

A sequence

`order_reference_seq` produces `DEL-0001`. A sequence rather than a row count, so references are stable and two backend processes cannot generate the same one.

6. A contradiction in the specification, resolved

§9.2 says show the "New supplier" badge when a supplier has **no completed orders yet**. §12.2 says give the neutral score and badge to suppliers with **fewer than three completed orders**.

Those disagree. **Three** was chosen, because §12.2 is the section that actually reasons about the failure mode: a supplier with one completed order has a score built on a single rating, which is exactly the case the badge exists to warn about. §9.2 states the rule in passing without arguing for it.

The number is stored in `app_config` as `new_supplier_min_orders`, so it can be retuned once real ratings exist without touching code. Recorded as **D-008**.

7. What was run

No part of this touched your Supabase project, because there isn't one yet. Everything ran locally.

#	Command	Purpose	Result
1	<code>docker --version</code> / <code>psql --version</code>	Look for an existing Postgres	Neither installed
2	<code>pip install pgserver</code>	Try an embedded Postgres	No Windows build
3	<code>pip install sqlglot</code>	Parse-check the SQL	Installed
4	parse all 31 files	Catch syntax errors early	0 errors
5	<code>curl ... postgresql-16.4-1-windows-x64-binaries.zip</code>	A real Postgres to run against	339 MB
6	<code>unzip -qt pg.zip</code>	Confirm the download completed	Archive intact
7	<code>initdb -D pgdata -U postgres</code>	Create a throwaway data directory	Success
8	<code>pg_ctl start -o "-p 54833"</code>	Start on a non-default port	Listening
9	<code>createdb inventix</code>	Empty target database	Created
10	apply <code>supabase_shim.sql</code>	Local stand-in for Supabase internals	Applied
11	apply all 30 schema files in order	The actual test	All 30 ok
12	apply <code>seeds/demo_data.sql</code>	The file the harness had skipped	Loaded
13	run <code>rls_test.sql</code>	The six-step security procedure, for real	All pass
14	run constraint tests	The five non-spec rules	All pass
15	<code>pg_ctl -m fast stop</code>	Shut the test server down	Stopped

About the shim

`supabase_shim.sql` is **not part of the project** and is not in `database/`. It exists only so a plain PostgreSQL can stand in for Supabase during verification. It creates three things Supabase already provides:

- the `auth` schema and `auth.users`, because `profiles.id` references it
- `auth.uid()`, which on Supabase reads the caller's id from their JWT — locally it reads a session variable instead, which is what makes it possible to impersonate one user and then another
- the `anon`, `authenticated` and `service_role` roles, with `service_role` marked `BYPASSRLS`

That last detail is worth seeing written down: **`BYPASSRLS` is a PostgreSQL role attribute, not a Supabase convention.** Every policy in `policies/` is simply skipped for `service_role`. That is the entire reason its key must never leave the backend.

8. What the verification proved

Every file applies, in order

All 16 migrations, both functions, all 9 policy files and all 4 seeds executed with `ON_ERROR_STOP=1`, meaning any error would have halted the run. None did.

Row level security actually works

The six steps from `docs/09-database-build.md` Part 5, run against real rows:

Check	Result
Customer B sees customer A's stock	0 rows — pass
Supplier sees any stock at all	0 rows — pass
Customer A sees their own stock (control)	1 row — pass
Customer B promotes themselves to supplier	refused — pass
Supplier creates a listing owned by someone else	refused — pass
Any client inserts its own <code>profiles</code> row	refused — pass

The control row matters as much as the others. Three "0 rows" results would also be produced by a schema where nobody can read anything, so proving A *can* see their own data is what makes the other results mean something.

Step 4 — the self-promotion attempt — is the one that is easy to skip and the one that matters most. A customer who can set `role = 'supplier'` can then read every listing, every order and every rating in the system. It is refused by the `with check` clause on `profiles_update_own`.

The functions do what they claim

Check	Result
<code>apply_stock_adjustment(item, -15, ...)</code> on a quantity of 100	returned 85, wrote exactly 1 audit row — pass
The same function asked to go below zero	refused — pass
<code>recompute_supplier_ranking()</code>	wrote 1 row — pass
A supplier with no ratings at all	scored 60.00 , not 0 — pass

That last line is §12.2 working. A new supplier scoring zero would sit at the bottom of every search permanently and could never earn the first order that would give them a rating.

The demo data loads and behaves

`seeds/demo_data.sql` produced 5 profiles, 5 stock items, 4 listings, 2 orders, 2 order lines, 1 rating and 4 adjustments. References generated themselves as `DEL-0001` and `DEL-0002`. The ranking job then scored the

rated supplier **81.0** (one five-star rating, three-day measured delivery) and the unrated one **60.0** — and marked both `is_new_supplier`, correctly, since both are below three completed orders.

The added constraints hold

Check	Result
<code>updated_at</code> changes on UPDATE	pass — the trigger fires
The same <code>idempotency_key</code> used twice	refused — pass
An order rejected with no reason	refused — pass
A customer given <code>delivery_areas</code>	refused — pass
A listing owned by a customer	refused — pass

One table is deny-all, on purpose

`app_config` has row level security enabled and **zero policies**, which in PostgreSQL means nobody gets anything. That is deliberate: nothing in the app needs to read the ranking weights, and exposing the tuning would tell a supplier how to game their own score. The backend reads it with the service key.

Every other table has RLS enabled *and* at least one policy. A table with RLS off would have been readable by anyone holding the anon key — which ships inside the app — so the harness checks for that explicitly and found none.

9. What was not done

Honest gaps, so none of these are a surprise later.

Nothing is applied to Supabase. There is no project, so there are no keys and no live database. Section 10 is the whole of what remains.

`stock_items.owner_id` has no role guard. `supplier_listings` got a trigger asserting its owner is a supplier; the equivalent check that a stock item's owner is a *customer* is not there yet. Low risk, since only the backend writes stock items, but it is an asymmetry worth closing.

No `catalog_requests` table. §5.2 and open question Q1 describe users submitting a product that creates a pending catalog entry for review. Both add-product screens already show the button and it does nothing. Modelling that needs Q1 answered first — specifically who reviews the queue.

`sales_uploads.column_mapping` is `jsonb` with no shape. It stores which column held the product name, the quantity and the date. A three-column table would be more rigid; `jsonb` was chosen because the mapping is read and written whole and never queried by key.

The `0016` numbering has a consequence. `docs/09-database-build.md` and `database/README.md` were written when there were fifteen migrations. Both are updated, but if anyone is working from a printed copy of the earlier PDF it will say `0001-0015`.

Seasonal dates are for 2026 only. Awurudu is fixed, but Ramadan moves about eleven days earlier each year and Vesak follows the lunar calendar. `seeds/seasonal_events.sql` needs refreshing annually — a stale row

produces a warning at the wrong time, which is worse than no warning.

10. How to apply it to Supabase

Nothing here has run against a real project. When you make one:

1. Create the project. Pick **Singapore** or **Mumbai** — the region sets every request's latency and cannot be changed later without recreating the project.
2. Save the database password. It is shown once.
3. From **Project Settings** → **API**, take the Project URL, the `anon` key and the `service_role` key. The first two go in `frontend/.env`; the `service_role` key goes in `backend/.env` **only** and never anywhere near the app.
4. In the SQL Editor, run in this order: - `migrations/0001` through `0016`, in numeric order — foreign keys refer backwards - `functions/` — both files - `policies/profiles.sql` **first**, because it defines `auth_role()` which the others call, then the remaining eight - `seeds/app_config.sql`, `seeds/product_catalog.sql`, `seeds/seasonal_events.sql`
5. For a demo: create two auth users in the dashboard, paste their UUIDs into the top of `seeds/demo_data.sql`, then run it. The file refuses to run if you forget.
6. Re-run the step 4 check from section 8 against the live project. Passing locally is good evidence, not proof — Supabase applies its own grants on top.
7. Regenerate the frontend types and diff them against the hand-written file:

```
npx supabase gen types typescript --project-id <id> > frontend/src/types/database.ts
```

Step 7 matters more than it looks. `frontend/src/types/database.ts` was written from \$5 *before* this schema existed. Where they disagree, nothing errors — the field simply arrives `undefined` on a screen, which is the most expensive kind of bug to find.

11. Documents updated

File	Change
<code>docs/04-decision-log.md</code>	Added D-006 (the new table), D-007 (three columns), D-008 (badge threshold)
<code>docs/05-changelog.md</code>	Phase 1a entry
<code>database/README.md</code>	The apply order was missing the <code>functions/</code> step, and said <code>0015</code>
<code>database/ERD.md</code>	<code>supplier_ranking</code> added to the map and the relationship table

12. Where this leaves the project

The frontend runs on a phone against fixture data. The database exists as SQL and is proven to apply cleanly. Nothing connects them yet, because the backend does not exist and no Supabase project has been created.

The next piece is `backend/app/core/` and the `shared/auth` feed, which is the first thing that needs both halves at once — and `docs/08-api-contract.md` already says exactly what it has to do.